

Embedded C

- "To write clean, efficient firmware for your STM32 ELF binaries, you must master specific Embedded C concepts.
- Unlike standard desktop C, Embedded C directly manipulates hardware registers, manages strict memory boundaries, and handles asynchronous hardware events.
- The essential Embedded C topics are organized below by practical application.

1. Hardware Register Access & Memory Mapping

- Because you are working without an OS, your C code must talk directly to physical memory addresses to control peripherals (like GPIO, UART, or Timers).
 - Pointers to Hardware: Casting absolute hex memory addresses into volatile pointers to read or write to hardware registers.
 - Bitwise Operations: Mastering mask operations (&, |, ~, ^, <<, >>) to flip specific bits in a control register without altering surrounding bits.
 - Structure Mapping: Creating customized C struct types where layout fields match the exact sequential hardware register offsets of the STM32 peripheral.

2. Embedded Storage Modifiers

- Using standard C variable declarations without modifiers can cause the compiler to over-optimize your code, breaking hardware functionality.
 - volatile Keyword: Tells the compiler that a variable's value can change outside the local program flow. It forces the CPU to read the actual RAM or register address every single time instead of using a cached CPU register. Mandatory for hardware registers and variables shared with interrupt handlers.
 - const Placement: Forces variables into the .rodata Flash section of your ELF file instead of RAM, saving precious runtime memory.
 - static Scope: Restricts variable or function visibility strictly to its own file or function block, maintaining encapsulation in modular driver code.

3. Interrupts & Asynchronous Execution

- STM32 chips use a Nested Vectored Interrupt Controller (NVIC) to pause your main program and react instantly to hardware signals.
 - Interrupt Service Routines (ISRs): Writing specialized, ultra-fast C functions that map directly to the addresses inside your startup.s vector table.
 - Critical Sections: Temporarily disabling interrupts when modifying shared resources to prevent data corruption.
 - Atomicity: Understanding that multi-step operations (like incrementing a 32-bit variable on an 8-bit core, or non-atomic reads) can be split in half by an interrupt.

4. Memory Layout & Custom Allocation

- Since you lack an OS memory manager like malloc(), you must explicitly control code placement.
 - Linker Symbols: Importing variables declared in your .ld linker script (like _sdata or _sbss) straight into C using extern.
 - Compiler Attributes: Using **attribute((packed))** to prevent the compiler from inserting padding bytes inside your communication structs, or

attribute((section(".ccmram")))) to force specific variables or functions into fast core-coupled RAM.

5. Bare-Metal Architecture Patterns

- Writing maintainable firmware requires structuring how your code executes over time.
 - Super-Loop (While-1): The foundational bare-metal setup consisting of an initialization block followed by an infinite loop executing serial state machines.
 - State Machines: Using switch-case blocks and enum tracking to run complex logic sequentially without blocking execution.
 - Polling vs. Interrupts: Balancing when to loop-wait for a hardware flag versus letting hardware fire an interrupt asynchronously.

Components of a Toolchain

1. Preprocessor

- Processes directives such as #include, #define by replacing them with their corresponding value

2. Compiler

- Converts C code into assembly language.
- Assigns addresses(logical) to the functions and variables
- Divides code into multiple sections like .text, .data, .bss, .rodata, .symtab, .symstr, etc.

3. Assembler

- Converts assembly code into object code.

4. Linker

- Combines:
 - Object files
 - Libraries
 - Startup code
- Produces executable(.o + Libraries → .elf).

5. Object Copy Utility

- Extracts binary image (.elf to .bin or .elf to .hex)

6. Debugger

- Used for:
 - Breakpoints
 - Single stepping
 - Variable inspection

7. Flash Programmer

- Loads firmware into target MCU.
- Examples:
 - ST-Link
 - J-Link
 - avrdude

GNU ARM Toolchain Example

Popular toolchain for ARM Cortex-M MCUs.

- arm-none-eabi-gcc : Compiler
- arm-none-eabi-as : Assembler
- arm-none-eabi-ld : Linker

- arm-none-eabi-objcopy : Generate .hex/.bin
- arm-none-eabi-gdb : Debugger

Cross Toolchain

- In embedded systems:
 - Development machine = Host
 - Microcontroller = Target
- The generated code runs on a different CPU architecture.
- Example:
 - Host : x86 Linux PC
 - Target : ARM Cortex-M4
- Example: STM32 Development
 - main.c -> arm-none-eabi-gcc -> app.elf -> objcopy -> app.hex -> ST-Link -> STM32 MCU

Sections of an ELF File

- An ELF (Executable and Linkable Format) file contains several sections used by the linker, loader, and debugger.
- In embedded Linux and embedded systems, understanding ELF sections is important for analyzing memory usage and program layout.

```

+-----+
| ELF Header |
+-----+
| .text      |
+-----+
| .rodata    |
+-----+
| .data      |
+-----+
| .bss       |
+-----+
| .init_array|
+-----+
| .fini_array|
+-----+
| .symtab    |
+-----+
| .strtab    |
+-----+
| Debug Sections |
+-----+

```

1. .text Section

- Contains:
 - Executable machine instructions
 - Program code

- Functions

2. .rodata Section

- Read-only data.
- Contains:
 - String literals
 - Constant data

3. .data Section

- Initialized global and static variables.

4. .bss Section

- Uninitialized global and static variables.

5. Heap (not present in elf file)

- Dynamic memory allocation.

6. Stack (not present in elf file)

- Function calls and local variables.

7. .symtab

- Symbol Table
- Contains:
 - Function names
 - Variable names
 - Addresses

8. .strtab

- String Table
- Stores symbol names.

9. .init_array

- Contains constructors executed before main().
- Execution order:
 - Reset → Startup → .init_array → main()

10. .fini_array

- Contains destructors executed after main() exits.
- Execution order:
 - main() → .fini_array → Program Exit

11. Debug Sections

- Examples:
 - .debug_info
 - .debug_line
 - .debug_abbrev
 - .debug_str

Embedded Memory Mapping

- Typical embedded system:
 - Flash
 - .text
 - .rodata
 - .data (initial values)
 - RAM
 - .data (runtime copy)

- .bss
- Heap
- Stack

SUNBEAM